Romi Lesson 20 – Subsystem Gripper

Now we are ready to create our first code for a new mechanism. The programmer's function is to bring 'life' to a mechanical system. The design team has designed a drivebase, an arm, a shooter or some system that the robot needs to score points. The build team has taken that design from drawings to working parts. Your job is to take the driver's controls and connect those controls to the mechanisms so they can be safely operated and score points to, hopefully, win matches.

In this lesson you will continue to work on your branch of the RomBot2026 code so make sure that you have fetched your latest copy using GitHub desktop.

The gripper that is attached to the Romi consists of two arms driven by a servo. A servo is a device consisting of a motor, a position sensor, and electronics for control. The motor is driven by a PWM signal just like the Spark motors. The difference is that instead of the signal controlling the speed of the motor, it controls the final position. The motor electronics uses a feedback system to control the motor to keep it at that position. (You will learn more about feedback systems using PIDs later.

By now you have realized that we need a subsystem for the gripper and a command to set the gripper to a position. This is very similar to controlling an LED like we did previously.

Start by opening your RomiBot2026 branch in VSCode. Right click on subsystems - Create a new class/command - Subsystem. Give it the name 'GripperSubsystem' and press enter. Now you will have an empty subsystem listed under subsystems.

Remember that the subsystem is where all the methods relating to the gripper (and therefore the servo used in the gripper) are defined. We need to define minimum and maximum values for the servo so that it doesn't jam and damage itself. We need a method for setting the arm position and it is good practice to also put in a method reading the position of the servo.

If you are adventurous, you are welcome to write this subsystem code, but It is available here if you want to copy and paste.

GripperSubsystem

Exercise 1: Add a command GripperSet which will set the gripper to a position passed as a parameter. Look at LEDSet for ideas.

Now you have a command and subsystem and if you try to run it, error messages.

In Constants you have to add the gripper constants that are referenced.

```
// Gripper constants

    public static final class GripperConstants {

      public static final int kServo = 3;

      public static final double gripperMin = 0.3;

       // This sets how wide the gripper can open. 0.3 for most applications - 0.0 is fully
open

      public static final double gripperMax = 0.6; // 0.65 onRomi-2,3,10,11: 0.6 nominally

    }
```

In RobotContainer you have to define the subsystem. In the section where it says ‘Define robot subsystems’ add:

```
  private final GripperSubsystem gripperSubsystem = new
GripperSubsystem(Constants.GripperConstants.kServo);
```

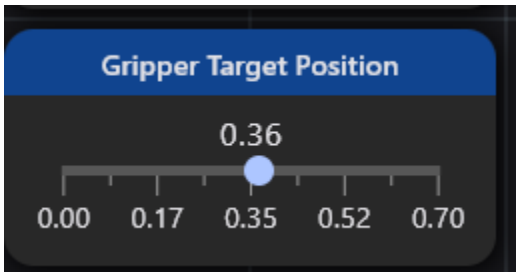
Also, add the gripper controls to the SmartDashboard.

```
//  GRIPPER commands
    //These are extremes. Actuals will be determined by gripperMax and gripperMin in
Constants.GripperConstants
    SmartDashboard.putData("Gripper Open", new GripperSet(gripperSubsystem  , 0.0));
     SmartDashboard.putData("Gripper Close", new
GripperSet(gripperSubsystem , 1.0));
```

You will also have to add an ‘import’ statement to import the GripperSubsystem.

Don’t forget to add widgets to Elastic.
Run and test.

Exercise 2: Add to Elastic a widget to enter a value so you can set the gripper to any desired position.



This will be needed for the upcoming Romi Rodeo. **TBD**